

AGILE PROJECT DOCUMENTATION

Project Name: Career Match AI

Project Title:

*AI Career Companion Agent for Internship
Matching and Interview Preparation*

Prepared By: Madireddy Meena Mahanth Harinadh

Project Duration: July 22, 2026 – September 10, 2026

Mentor: Mr. Vijay Parmar

Index

No.	Section	Page No.
1	Project Overview	3
2	Project Objective	4
3	Technology Stack	5
4	Project Requirements	7
5	System Architecture / Workflow	8
6	Product Backlog	11
7	Sprint Planning	13
8	Sprint Tasks & Status	15
9	Implementation / Key Features	16
10	Testing	20
11	Challenges & Solutions	22
12	Final Project Status	23
	Limitations	24
	Future Scope	25
	Conclusion	26

1. Project Overview

1.1 Project Name

Career Match AI

1.2 What is the Project?

Career Match AI is an AI-powered web application built for the Infosys AI internship program. It combines four capabilities behind a single authenticated experience: (1) resume upload and AI-assisted structured data extraction, (2) semantic matching of a candidate's profile against a set of internship postings using vector similarity search, (3) an AI-driven mock interview agent that asks role-specific questions and scores each answer, and (4) a document question-answering feature that lets a user upload a PDF and ask questions grounded strictly in that document's content. The current system uses a React 19 + TypeScript + Vite single-page frontend, a FastAPI backend, PostgreSQL (production) for structured relational data, FAISS indexes for semantic retrieval, the Groq API for language generation and evaluation, and Sentence-Transformers (all-MiniLM-L6-v2) for generating embeddings. The frontend is deployed on Vercel and the backend on Render (production configuration; deployment verification pending).

1.3 Problem Statement

Internship seekers commonly face several practical difficulties: it is time-consuming to manually compare a resume against many internship listings to judge fit; candidates often do not have a structured, searchable record of their own skills, projects, and experience; and many candidates lack easy access to realistic, role-specific interview practice with actionable feedback. Career Match AI addresses these problems by automatically extracting structured information from an uploaded resume, using semantic search to surface internships that are conceptually related to the candidate's skills and experience (rather than requiring exact keyword matches), and providing an AI interviewer that asks questions and scores responses. The system also allows a user to upload any PDF and ask questions about it, answered strictly from that document's content.

NOTE

The application presents matches, scores, and interview feedback as AI-generated decision-support output. It does not guarantee interview selection, internship placement, or employment outcomes.

1.4 Target Users

- Students
- Internship seekers
- Early-career candidates preparing for technical or behavioral interviews

2. Project Objective

The project was built to achieve the following objectives:

- Automate structured extraction of candidate information (skills, education, work experience, and projects) from an uploaded PDF resume using a large language model (Groq API).
- Match a candidate against a set of internship postings using semantic (embedding-based) retrieval rather than plain keyword search.
- Provide AI-assisted, role-specific mock interview practice with per-answer scoring and written feedback, followed by a dynamically generated next question.
- Enable document-grounded question answering, so a user can upload a PDF and receive answers based only on that document's content.
- Combine a relational database (PostgreSQL in production) as the structured source of truth with FAISS vector indexes as a dedicated semantic retrieval layer, rather than forcing one store to do both jobs. (The original development build used MySQL, ChromaDB and Google Gemini; see Section 3.)

3. Technology Stack

The table below lists the technologies of the current Career Match AI stack.

Category	Technology	Purpose
Programming Language	Python 3	Backend application logic, AI services, and data access
Frontend Language / Tooling	TypeScript, Vite	Type-safe frontend code and fast development/build tooling
Frontend Framework	React 19	Single-page application UI
Routing	React Router v7	Client-side page routing and protected routes
Styling	Tailwind CSS	Utility-first styling of the frontend
HTTP Client	Axios	Frontend-to-backend API requests, with a JWT bearer token attached via an interceptor
Icons	Lucide React	UI iconography across the frontend pages
Backend Framework	FastAPI	REST API layer exposing auth, profile, resume, matching, interview, and document endpoints
ORM	SQLAlchemy 2.x	Object-relational mapping (PostgreSQL in production; SQLite as local development fallback)
Migrations	Alembic	Configured for schema migrations (see note below)
Config / Validation	Pydantic / pydantic-settings	Environment-based settings and request/response schema validation
Database	PostgreSQL (cloud / managed) for production	Production relational source of truth for users, profiles, resumes, internships, matches, documents, and interview sessions
Vector Search	FAISS	Vector similarity search over internship postings and user-uploaded document chunks
Embeddings	Sentence-Transformers ("all-MiniLM-L6-v2")	Converts resume text, internship text, and document chunks into

Category	Technology	Purpose
		vector embeddings
AI / LLM	Groq API	Structured resume extraction, interview question generation and scoring, match explanations, and document Q&A
Authentication	JWT (python-jose) + Passlib (bcrypt)	Token-based session authentication and password hashing
Document Processing	PyPDF2	Text extraction from uploaded PDF resumes and documents
Testing	pytest + FastAPI TestClient	Automated backend tests (currently covering authentication)
Frontend Hosting	Vercel	Production frontend hosting (verification pending)
Backend Hosting	Render	Production API hosting (verification pending)
Monitoring	UptimeRobot	Uptime monitoring via /health (not confirmed as configured)

NOTE

Note: the frontend's package.json also lists "recharts", but no component imports or renders it, so it is not listed as in use. Historical note: the original build used MySQL, ChromaDB and Google Gemini; they were replaced by PostgreSQL, FAISS and Groq. Hosting and monitoring are described in Section 5.1.

NOTE

Note on Alembic: an Alembic configuration exists, but the reviewed snapshot had no migration version files under alembic/versions; tables are created via Base.metadata.create_all. Re-verify against the current repository.

4. Project Requirements

4.1 Functional Requirements

- User registration and login using email and password (JWT-based session authentication).
- Retrieval and update of the current user's profile (name, phone, location, headline, summary, LinkedIn/GitHub/portfolio links, preferred roles and locations).
- Resume upload; the upload endpoint accepts .pdf and .docx file extensions, but text is currently extracted only from PDF files — a .docx upload is stored but is not parsed for content in the current implementation.
- AI-based structured extraction of resume data (name, contact details, summary, skills, education, experience, projects) using the Groq LLM, saved to the relational database and used to auto-fill empty profile fields.
- Semantic internship matching: the active resume is converted to a text summary, embedded, and compared against a FAISS index of internship postings to retrieve the top matches.
- AI-generated match explanations, and matched/missing skill lists, produced by the Groq LLM and persisted per match.
- AI mock interview sessions: a user selects a target role and difficulty, and the system starts a session, asks an opening question, and continues with an evaluate-then-ask-next-question loop.
- Per-answer interview scoring (0–10) and written feedback, generated by the Groq LLM and stored alongside each answer.
- Document upload (PDF only) and chunk-based indexing into a FAISS index for later retrieval.
- Document-grounded question answering: a user's question is answered using only the chunks retrieved from that user's own uploaded document(s).

4.2 Non-Functional Requirements

- Security: passwords are hashed with bcrypt; API access requires a valid JWT bearer token (7-day expiry).
- User data isolation: resumes, documents, matches, and interview sessions are filtered by the authenticated user's ID; document retrieval returns only the requesting user's own chunks.
- Error handling: the API returns structured HTTP errors (e.g., 400 for a duplicate registration or invalid file type, 404 for a missing interview session or document, 403 for an invalid token).
- Configuration management: secrets and endpoints (database URL, JWT secret, Groq API key, vector index location, upload directory) are read from environment variables.
- Maintainability: the backend is organized into clear layers — API routes, Pydantic schemas, SQLAlchemy models, and services — separating request handling from business logic.

NOTE

CORS is currently configured to allow all origins (allow_origins=[""]), which the source code itself comments is intended for development and should be restricted before production use.*

5. System Architecture / Workflow

This section describes the architecture, components, and data flow of each AI-driven feature, updated for the current Groq / PostgreSQL / FAISS stack.

5.1 High-Level Architecture

At a high level, the system follows a standard client-API-data-and-AI-services architecture:

- User → React Frontend (Vite, TypeScript, Tailwind, Axios) → FastAPI Backend
- FastAPI Backend → PostgreSQL (structured application data via SQLAlchemy; SQLite may be used as a local development fallback)
- FastAPI Backend → FAISS (semantic vector search for internships and documents)
- FastAPI Backend → Sentence-Transformers (local embedding generation)
- FastAPI Backend → Groq API (resume extraction, match explanations, interview evaluation, document Q&A)

The frontend communicates with the backend exclusively over REST endpoints prefixed with `/api/v1`, grouped into `auth`, `profile`, `resumes`, `matching`, `interview`, and `documents` routers. Authentication uses an OAuth2-compatible password flow: the frontend logs in against `/api/v1/auth/login` and attaches the returned bearer token via an Axios interceptor. Production configuration: User → Vercel React frontend → (HTTPS API) → Render FastAPI backend → Cloud PostgreSQL and Groq API, with FAISS and Sentence-Transformers in the retrieval layer. A single FastAPI application exposes a lightweight GET `/health` endpoint for Render health checks; UptimeRobot can monitor <https://YOUR-RENDER-SERVICE.onrender.com/health> (deployment and monitor setup pending verification).

5.2 Resume Processing Workflow

The documented flow, based on `app/api/routes/resume.py` and `app/services/resume_parser.py` (LLM provider updated to Groq), is:

- User uploads a file (.pdf or .docx accepted at the API level) through the Resume page.
- The file is saved to the server's configured upload directory.
- If the file is a PDF, its text is extracted with PyPDF2 (`extract_text_from_pdf`). If it is a .docx, no text extraction is currently performed — an empty string is passed forward.
- The extracted text is sent to the Groq LLM with a prompt instructing it to return a structured JSON object matching a fixed schema (name, contact details, summary, skills, education, experience, projects) and explicitly told not to invent information.
- The JSON response is parsed and validated against a Pydantic schema (`ResumeExtractionSchema`).
- The parsed resume, along with its structured skills, education entries, experience entries, and projects, is saved to the relational database (PostgreSQL in production; `resumes`, `resume_skills`, `resume_educations`, `resume_experiences`, `resume_projects` tables); any previous active resume for the user is deactivated.
- Empty profile fields (name, phone, LinkedIn, GitHub, summary) are automatically filled in from the parsed resume data if they were not already set.

NOTE

The final implementation currently performs PDF text extraction and structured extraction end-to-end. DOCX files are accepted by the upload endpoint but are not yet parsed for text content.

5.3 Internship Matching Workflow

The documented flow, based on `app/services/matching_service.py` and `app/services/embedding_service.py` (retrieval updated to FAISS, LLM to Groq), is:

- The candidate's active resume is retrieved from the database.
- A plain-text candidate summary is built by concatenating the skill names, experience descriptions/titles, and project descriptions/technologies from the parsed resume.
- This text is embedded using the Sentence-Transformers `all-MiniLM-L6-v2` model.
- The embedding is used to query the FAISS internships index for the 5 nearest internship postings by L2 distance (top-k carried over from the original build).
- Each result's raw distance is converted to a similarity score with the formula `similarity_score = max(0, 100 - distance × 50)`, as found in the original source code (carried over to the FAISS-based build).
- For each matched internship, the Groq LLM is asked to compare the candidate text against the internship's requirements and produce a short explanation plus lists of matched and missing skills, based only on the given text.
- The match (scores, explanation, matched/missing skills) is saved to the matches table and returned to the frontend.

NOTE

The internship postings currently searched against are the 5 demo internships created by `backend/scripts/seed_database.py` (Data Science Intern, Backend Developer Intern, Generative AI Intern, Frontend Developer Intern, and Data Engineering Intern) — a small, illustrative/demo dataset rather than a live or large-scale internship catalogue.

5.4 Interview Agent Workflow

The documented flow, based on `app/services/interview_agent.py` and `app/api/routes/interview.py` (LLM provider updated to Groq), is:

- The user starts a session by choosing a `target_role` and `difficulty`; a new `interview_sessions` row is created.
- The backend immediately calls the same evaluation function with a placeholder "Hello, I am ready to start the interview" message to generate the opening question, seeded with the candidate's resume text (trimmed to 2000 characters) and the target role/difficulty.
- The opening AI question is stored as an `interview_messages` row (`role = "ai"`).
- When the candidate submits an answer, a single Groq LLM call is made with the full conversation history and the new answer; the prompt asks the model to (1) score the answer 0–10 with concise feedback and (2) ask the next question, in one fixed-format response (`SCORE / FEEDBACK / NEXT_QUESTION` lines) that the backend parses.
- The user's answer, its score, and its feedback are saved as one message row; the next AI question is saved as a second message row.

- The frontend receives and displays the full running message list, including per-answer scores and feedback.

5.5 Document Q&A Workflow

The documented flow, based on `app/services/document_qa_agent.py` and `app/api/routes/document.py` (retrieval updated to FAISS, LLM to Groq), is:

- The upload endpoint accepts PDF files only (a non-PDF filename is rejected with an HTTP 400 error).
- The uploaded PDF's text is extracted with PyPDF2 and split into fixed-size chunks of 1,000 characters (simple character-count chunking, with no overlap or semantic boundary logic).
- Each chunk is embedded (Sentence-Transformers) and added to the FAISS documents index together with metadata identifying the owning `user_id`, the `document_id`, and the chunk index.
- When the user asks a question, the question is embedded and the FAISS index is queried for the 3 nearest chunks (top-k carried over), restricted to the requesting user's own document chunks.
- The retrieved chunk text is passed to the Groq LLM as context, with an explicit instruction to answer only from that context and to say so if the answer is not contained in it.
- If no LLM API key is configured, the backend still returns retrieved context (truncated) instead of a generated answer, rather than failing outright.

5.6 PostgreSQL vs. FAISS

The system deliberately uses two different databases for two different jobs, rather than using a single database for both:

PostgreSQL

Acts as the structured, relational source of truth (SQLite may serve as a local development fallback) for: users, profiles, resumes and their parsed data, internship postings, matches, documents (metadata), and interview sessions/messages.

FAISS

Acts as the semantic retrieval (vector similarity search) layer, together with Sentence-Transformers embeddings, for internship similarity search during matching and chunk-level retrieval for document Q&A. FAISS is a similarity-search library rather than a database, so it complements the relational store.

PostgreSQL	FAISS
Structured relational data	Vector (embedding) data
Transactional storage	Semantic similarity search
Source of truth	Retrieval / index layer
Relationships and constraints (foreign keys)	Embedding-based nearest-neighbour search

Using PostgreSQL for structured records keeps referential integrity (for example, a match always references a real user and a real internship). Using FAISS indexes for embeddings lets the system find conceptually similar content that a plain keyword search would miss. Combining the two avoids forcing either component to do a job it is not designed for.

6. Product Backlog

NOTE

This backlog reflects the final implementation's backlog/status, reconstructed from the source code. It is not a historical record of how or when each item was actually tracked during development, as no sprint board or issue tracker was provided in the project evidence.

ID	Feature / Task	Priority	Status
P001	Project setup (backend + frontend scaffolding)	High	Done
P002	Database schema and PostgreSQL setup (originally MySQL)	High	Done
P003	User authentication (register, login, JWT)	High	Done
P004	Profile management (get/update)	Medium	Done
P005	Resume upload endpoint	High	Done
P006	PDF text extraction	High	Done
P007	Groq-based structured resume parsing (originally Gemini)	High	Done
P008	DOCX resume text extraction	Medium	Pending
P009	Embedding generation (Sentence-Transformers)	High	Done
P010	FAISS integration (internships + documents indexes; originally ChromaDB)	High	Done
P011	Internship demo data seeding	Medium	Done
P012	Semantic internship matching + Groq explanation	High	Done
P013	AI interview agent (session, scoring, next question)	High	Done

P014	Document upload + chunked indexing	High	Done
P015	Document-grounded Q&A with user isolation	High	Done
P016	Frontend pages (Login, Register, Dashboard, Profile, Resume, Matching, Interview, Documents, Settings)	High	Done
P017	Automated backend testing	Medium	Partial — authentication only
P018	Project documentation	High	Done
P019	Production deployment and monitoring (Vercel frontend, Render backend, Cloud PostgreSQL, UptimeRobot via /health)	Medium	Pending verification

7. Sprint Planning

NOTE

Not available in the provided project evidence: no sprint board, issue tracker export, or dated commit history was supplied. The development phases below are reconstructed from the final implementation's structure and mapped onto the stated overall project duration (July 22 – September 10, 2026, approximately 7 weeks) for presentation purposes only; they do not represent a verified historical schedule. Historical development note: the sprint items below describe the original development stack (MySQL, ChromaDB, Google Gemini); the current implementation uses PostgreSQL, FAISS and Groq (Section 3).

Sprint 1 — Project Setup & Database

Goal: Establish the backend and frontend project structure and the MySQL schema.

- Set up FastAPI backend project structure (api/core/db/schemas/services)
- Set up React + TypeScript + Vite + Tailwind frontend project structure
- Define SQLAlchemy models for users, profiles, resumes, internships, matches, documents, and interview sessions
- Configure environment variables (.env) and Alembic

Sprint 2 — Authentication & Profile

Goal: Implement account creation, login, and profile management.

- Implement password hashing and JWT token issuance
- Implement register, login, and "current user" endpoints
- Implement profile get/update endpoints and the Profile page

Sprint 3 — Resume Processing

Goal: Implement resume upload and AI-based structured extraction.

- Implement PDF upload and PyPDF2 text extraction
- Integrate Gemini for structured resume field extraction
- Persist parsed skills, education, experience, and projects to MySQL

Sprint 4 — Semantic Matching (RAG Foundation)

Goal: Stand up the vector retrieval layer and internship matching.

- Integrate ChromaDB and Sentence-Transformers embeddings
- Seed demo internship data and index it into ChromaDB
- Implement matching_service.py and the /matching/internships endpoint

Sprint 5 — Interview Agent & Document Q&A

Goal: Implement the two remaining Gemini-driven features.

- Implement interview session creation and the evaluate/next-question loop
- Implement document upload, chunking, indexing, and grounded Q&A with per-user retrieval filtering

Sprint 6 — Frontend Integration & Testing

Goal: Connect all frontend pages to the backend and add automated tests.

- Build out Dashboard, Resume, Matching, Interview, Documents, and Settings pages
- Add automated authentication tests (tests/test_auth.py)
- Prepare final project documentation

8. Sprint Tasks & Status

NOTE

As with Section 7, per-sprint assignment and exact historical status changes are not available in the provided project evidence. The table below shows the status of each task organized under the reconstructed sprint labels from Section 7; items naming MySQL, ChromaDB or Gemini describe the original development stack, not the current one.

Sprint	Task	Status	Remarks
Sprint 1	Backend/frontend project scaffolding	Done	Reflects final structure; historical per-sprint tracking not available
Sprint 1	MySQL schema (SQLAlchemy models)	Done	—
Sprint 2	Authentication (register/login/JWT)	Done	—
Sprint 2	Profile management	Done	—
Sprint 3	PDF resume upload & text extraction	Done	DOCX extraction not implemented
Sprint 3	Gemini structured resume parsing	Done	—
Sprint 4	ChromaDB + Sentence-Transformers integration	Done	—
Sprint 4	Internship demo data seeding	Done	5 demo internships
Sprint 4	Semantic internship matching	Done	—
Sprint 5	AI interview agent	Done	—
Sprint 5	Document upload, chunking & Q&A	Done	PDF only
Sprint 6	Frontend page integration	Done	—
Sprint 6	Automated testing	In Progress	Only authentication tests present
Sprint 6	Final documentation	Done	This document

9. Implementation / Key Features

Feature 1 — Authentication

Description

Allows a candidate to create an account, log in, and access protected pages and API routes.

Implementation

Implemented with FastAPI's OAuth2PasswordRequestForm-based /auth/login endpoint, bcrypt password hashing via Passlib, and JWT access tokens (7-day expiry) verified on each protected request through a FastAPI dependency (get_current_user). The frontend AuthContext stores the token and attaches it to every Axios request.

Output

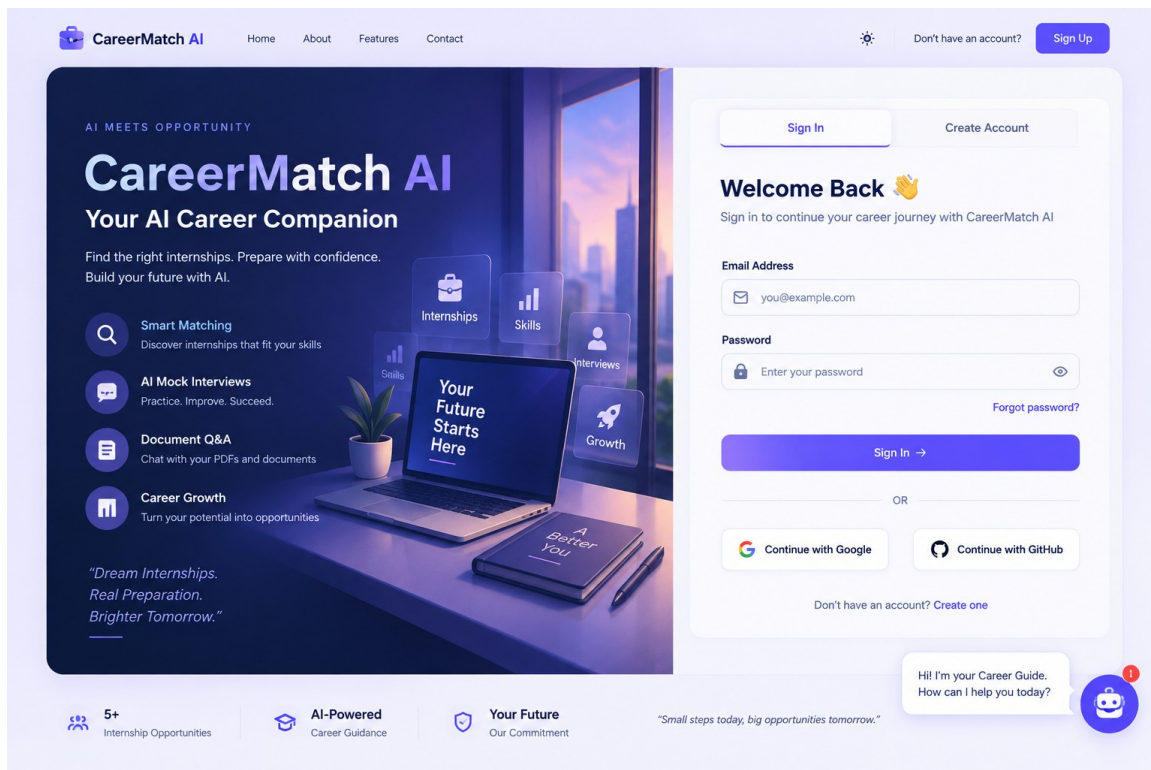


Figure 9.1 — Login / Registration Interface

Result

A user can register, log in, and receive a bearer token that grants access to the Dashboard and all other protected pages; invalid credentials are rejected with an HTTP 400 error.

Note on figures: Figures 9.1–9.6 are retained unchanged from an earlier UI iteration (purple/indigo theme). The current Career Match AI interface uses a warm orange / charcoal / cream identity: in dark mode, warm near-black/charcoal surfaces with an orange accent and subtle green status accents; in light mode, a warm cream background with white cards and an orange primary accent.

Feature 2 — Profile Management

Description

Lets a user view and edit their profile (name, phone, location, headline, summary, links, preferred roles/locations).

Implementation

A dedicated profiles table (one-to-one with users) is exposed through GET and PUT /profile endpoints. A profile row is auto-created on first access if one does not already exist. Fields left blank by the user are also auto-filled from resume parsing (see Feature 3).

Output

The screenshot displays the 'My Profile' page in the CareerMatch AI application. The page is divided into several sections:

- Header:** Includes the 'CareerMatch AI' logo, a search bar, and user navigation options.
- Left Sidebar:** Contains navigation links for Home, Profile (active), Resume, Internship Matching, Mock Interview, Documents, and Settings.
- Profile Section:** Features a profile picture, name 'Meena Mahanth Harinadh', title 'Computer Science Student', and a bio. It also shows contact information (email, phone, location) and a 'Profile Complete' status.
- Profile Completion:** A circular progress indicator shows 100% completion. A checklist on the right lists: Personal Information, Skills, Education, Work Experience, and Career Preferences, all marked as complete.
- Personal Information:** A form section with fields for Full Name, Email Address, Phone Number, and Location, all populated with the user's details.
- Bio / About Me:** A text area containing the user's bio: 'Passionate about AI/ML and software development. Eager to learn, contribute and grow through impactful internship opportunities.'
- Career Preferences:** A section with dropdown menus for 'Interested Domains' (Machine Learning, Data Science), 'Preferred Work Mode' (Either (Remote / On-site)), 'Preferred Locations' (Bangalore, Hyderabad, Remote), and 'Availability' (3-6 months).
- Skills:** A list of skills including Python, Machine Learning, Data Analysis, SQL, React, JavaScript, FastAPI, Problem Solving, and Communication.
- Education:** A section showing 'B.Tech in Computer Science and Engineering' from 'ABC Engineering College' for the years '2022 - 2026' with a status of 'Pursuing'.
- Work Experience:** A section with a message: 'No work experience added yet. Add your internship or work experience to get better matching.'
- Footer:** Includes a quote: 'Invest in your skills today for a brighter tomorrow.' and a user profile card for Meena Mahanth Student.

Figure 9.2 — Profile Page

Result

A user can view their profile and update it; fields populate automatically after a resume is parsed if they were previously empty.

Feature 3 — Resume Upload & Parsing

Description

Lets a user upload a resume and have its content converted into structured data.

Implementation

The /resumes/upload endpoint accepts .pdf and .docx files, but only PDF files are currently processed: text is extracted with PyPDF2, then sent to the Groq LLM with a schema-constrained prompt to extract name, contact details, summary, skills, education, experience, and projects as JSON. The parsed result is validated with Pydantic and saved across the resumes table and its related skill/education/experience/project tables.

Output

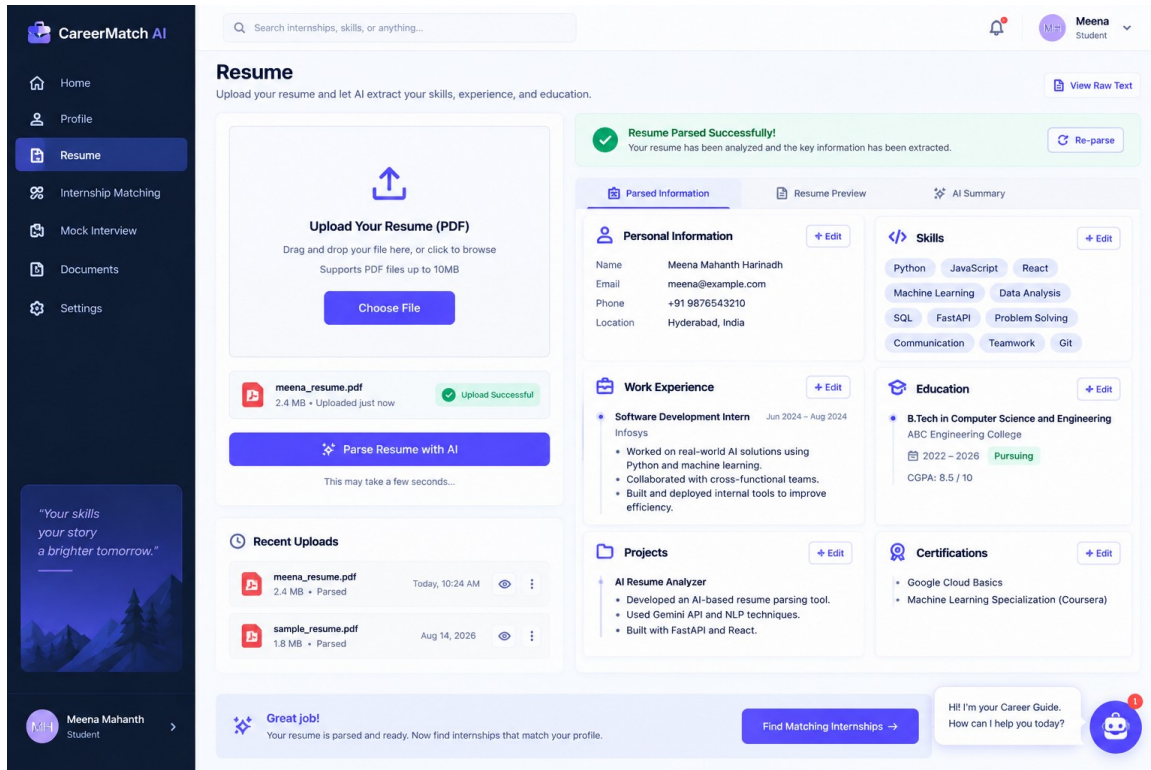


Figure 9.3 — Resume Upload and Parsed Data

Result

Uploading a PDF resume produces a structured, saved record of the candidate's skills, education, experience, and projects, and updates blank profile fields. DOCX uploads are currently accepted by the endpoint but are not text-extracted or parsed.

Feature 4 — Internship Matching

Description

Surfaces internships that are semantically relevant to the candidate's parsed resume.

Implementation

The candidate's skills/experience/projects are summarized into text, embedded with Sentence-Transformers, and compared against a FAISS index of internship postings using nearest-neighbour (L2 distance) search. Distances are converted to a 0–100 similarity score, and the Groq LLM generates a short explanation and matched/missing skill lists for each result, which are saved and returned to the frontend Matching page.

Output

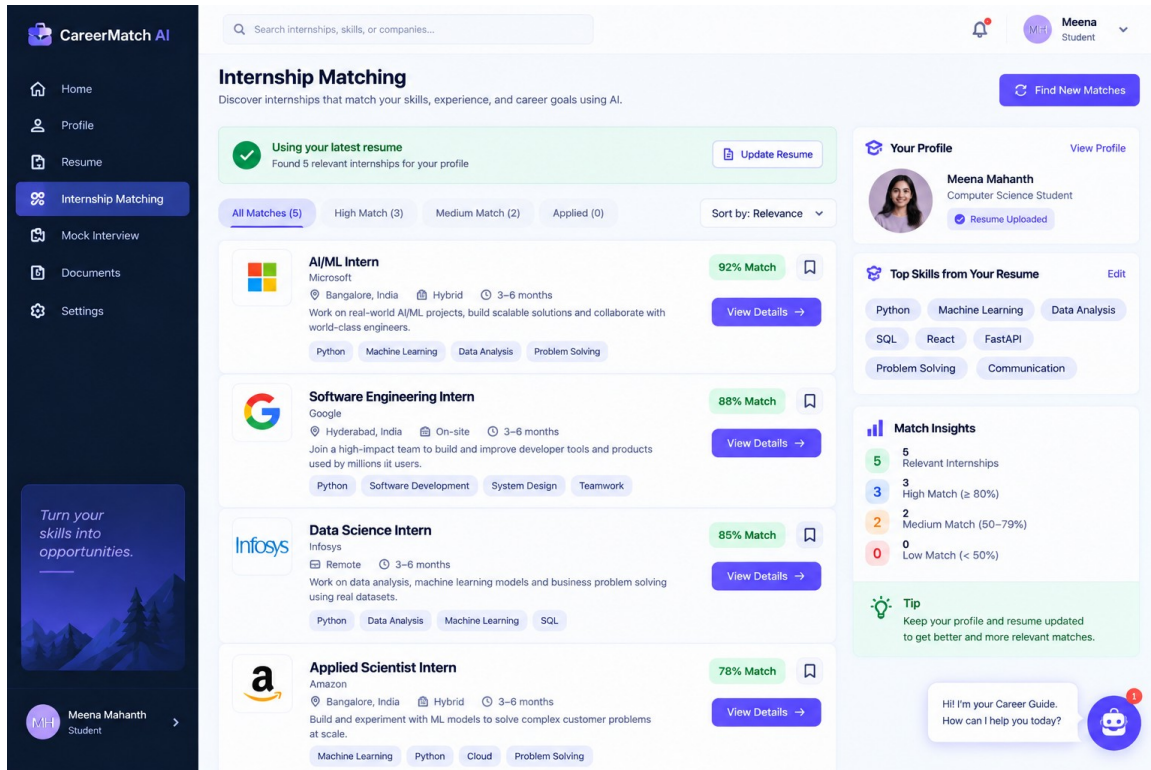


Figure 9.4 — Internship Matching Results

Result

A candidate receives up to 5 ranked internship matches (from the seeded demo dataset) with a similarity score, an AI-generated explanation, and matched/missing skill lists.

Feature 5 — AI Interview Agent

Description

Provides an AI-driven mock interview experience tailored to a chosen role and difficulty.

Implementation

A user starts a session with a target role and difficulty; the backend generates an opening question with the Groq LLM, seeded with the candidate's resume text. Each subsequent answer is evaluated in the same model call that produces the next question, following a fixed SCORE / FEEDBACK / NEXT_QUESTION response format that the backend parses and stores per message.

Output

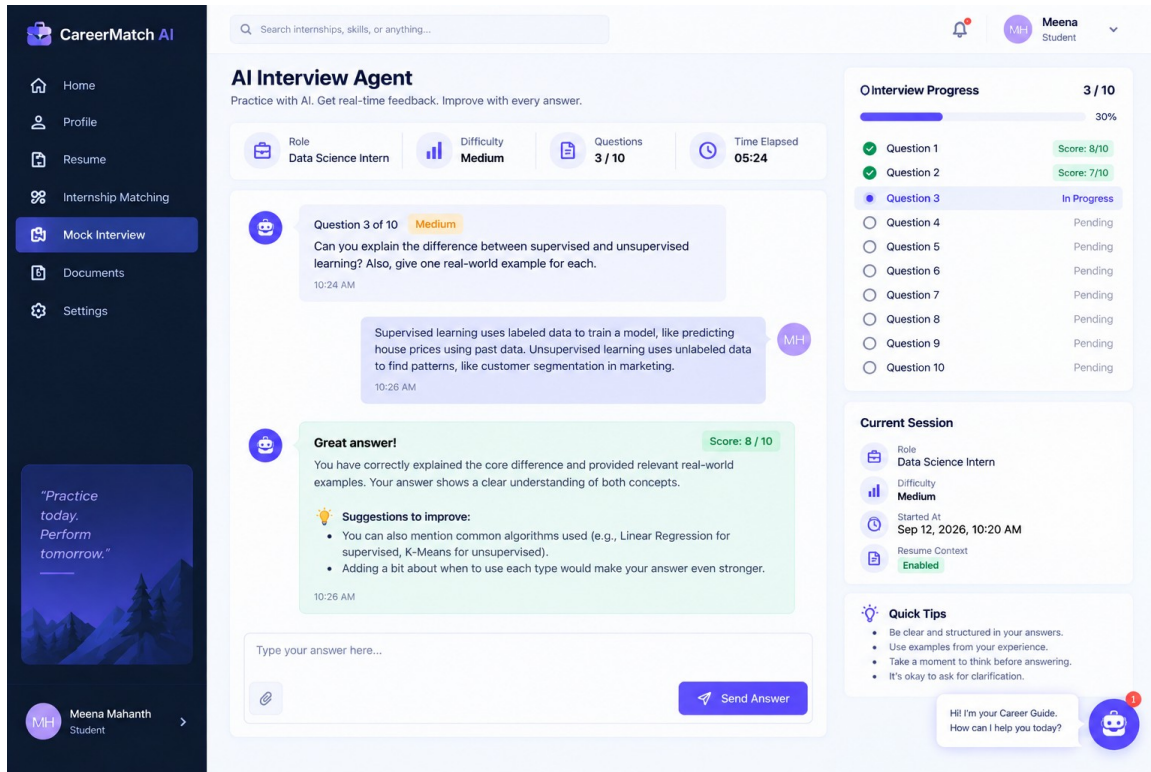


Figure 9.5 — AI Interview Agent

Result

A candidate can hold a multi-turn mock interview and receive a 0–10 score and written feedback after each answer, along with a new, contextually relevant question.

Feature 6 — Document Q&A

Description

Lets a user upload an arbitrary PDF document and ask questions answered only from that document.

Implementation

The `/documents/upload` endpoint accepts PDF files only, extracts text with PyPDF2, splits it into fixed 1,000-character chunks, embeds each chunk, and stores it in a FAISS documents index tagged with the owning user's ID. Questions are embedded and matched against the top 3 chunks belonging to that same user, and the Groq LLM is instructed to answer only from that retrieved context.

Output

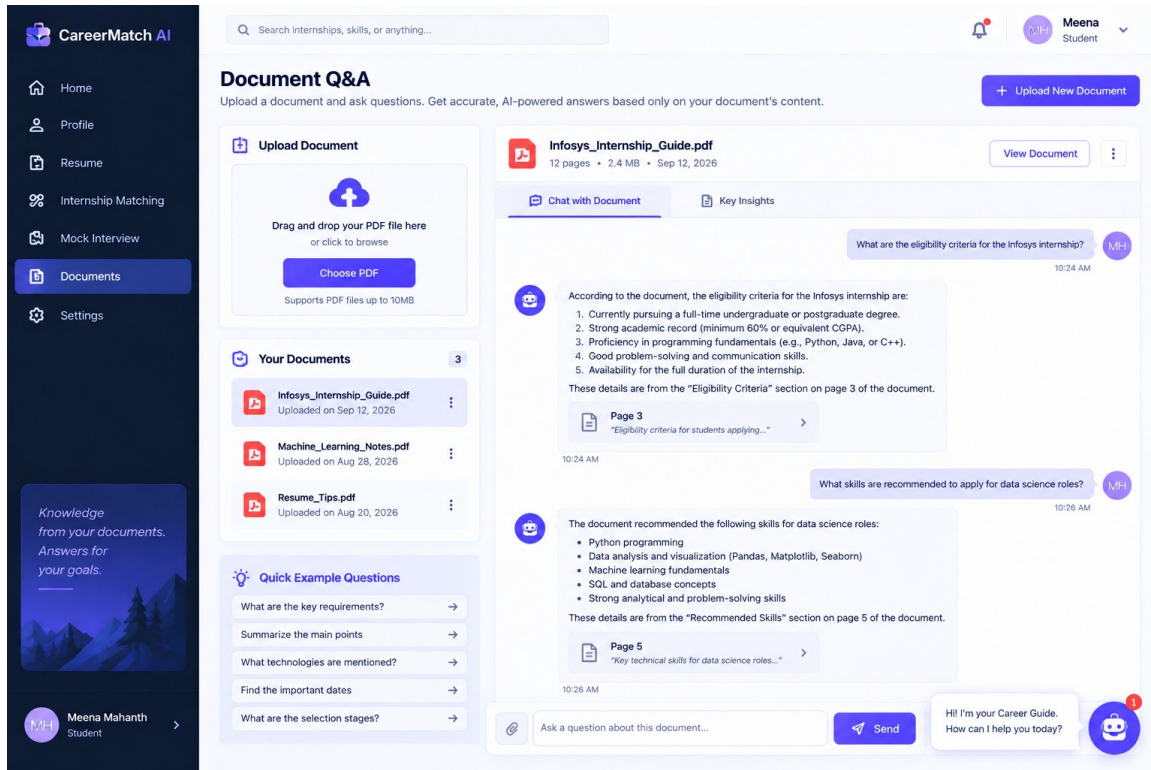


Figure 9.6 — Document Q&A

Result

A user can upload a PDF and ask questions about it, receiving answers grounded in the document's own text; if the retrieved context does not contain the answer, the system responds that the document does not contain enough information.

10. Testing

The repository includes one automated backend test file, tests/test_auth.py, run with pytest and FastAPI's TestClient, covering user registration and login. No automated tests were found for resume parsing, matching, the interview agent, or document Q&A. The table below lists both the existing automated test and the manually testable behaviors implied by the source code; results are marked "Not yet verified" wherever no executed test run or evidence was available.

Test ID	Test Case	Expected Result	Actual Result	Status
T001	Register a new user (POST /api/v1/auth/register)	User is created and returned (or a 400 error if the email already exists)	Automated pytest test present in tests/test_auth.py (test_register_user); not executed in the documentation environment (requires a configured database instance)	Not yet verified
T002	Log in with valid credentials (POST /api/v1/auth/login)	A 200 response containing an access_token	Automated pytest test present in tests/test_auth.py (test_login_user); not executed in the documentation environment	Not yet verified
T003	Upload a PDF resume and parse it	Resume text is extracted and structured fields are returned/saved	No automated test found for this endpoint	Not yet verified
T004	Upload a non-PDF/DOCX file as a resume	HTTP 400 error ("Only PDF and DOCX files are allowed")	Enforced directly in resume.py's file-extension check; no automated test found	Not yet verified
T005	Run internship matching for a candidate with an active resume	Up to 5 ranked internship matches with scores and explanations	No automated test found for this endpoint	Not yet verified
T006	Start and continue an interview session	Opening question generated; subsequent answers scored with	No automated test found for this endpoint	Not yet verified

Test ID	Test Case	Expected Result	Actual Result	Status
		feedback and a next question		
T007	Upload a PDF document and ask a grounded question	Answer generated using only retrieved document chunks	No automated test found for this endpoint	Not yet verified
T008	Upload a non-PDF file as a document	HTTP 400 error ("Only PDF files are allowed for documents currently")	Enforced directly in document.py's file-extension check; no automated test found	Not yet verified

10.1 AI / RAG Testing

The following AI/RAG-related behaviors are implemented in the source code but are not covered by any automated test in the provided repository:

- Skill-based internship matching (semantic similarity via Sentence-Transformers + FAISS)
- Resume-based structured data extraction accuracy (Groq LLM)
- Document-grounded Q&A correctness and refusal behavior when context is insufficient
- Interview question generation relevance and score consistency

NOTE

No accuracy percentages, benchmark scores, or measured evaluation metrics for the matching, extraction, or Q&A features were found in the project evidence, so none are claimed in this document.

11. Challenges & Solutions

NOTE

The challenges below are drawn directly from patterns and safeguards observed in the source code (fallback handling, explicit anti-hallucination instructions, per-user retrieval scoping, and the distance-to-score conversion). No project notes, issue tracker, or personal account of development challenges was included in the provided evidence, so no additional narrative challenges beyond what the code itself demonstrates are claimed here. Technology names reflect the current stack (Groq, FAISS); the safeguards were originally observed in the Gemini/ChromaDB build.

Challenge	Solution
Extracting structured information from unstructured resume text	Combined PyPDF2 text extraction with a schema-constrained Groq LLM prompt that explicitly instructs the model not to invent missing fields, and validated the model's JSON output against a Pydantic schema (ResumeExtractionSchema) before saving it.
Making internship search semantic rather than keyword-based	Used Sentence-Transformers to embed both candidate profile text and internship postings, and used FAISS nearest-neighbour search so that conceptually related skills/experience can match an internship even without exact keyword overlap.
Keeping a user's uploaded documents private from other users during retrieval	Scoped document retrieval to the requesting user only. The original build attached a user_id field to each chunk's ChromaDB metadata and applied a where={"user_id": ...} filter on every document-search query; the equivalent mechanism in the current FAISS-based retrieval should be re-verified against the code.
Turning a raw similarity distance into an interpretable score	Converted the L2 distance returned by vector search into a bounded similarity_score using the formula $\max(0, 100 - \text{distance} \times 50)$ in matching_service.py, so the frontend can display a simple 0–100 score instead of a raw distance value.
Application configuration relies on an LLM API key that may not always be present	Each LLM-dependent service function checks settings.LLM_API_KEY (the Groq API key in the current configuration) and returns a safe fallback (an empty extraction schema, a neutral match explanation, or a message asking the user to configure the key) instead of raising an unhandled error when the key is missing.

12. Final Project Status

12.1 Completed Features

- User registration, login, and JWT-based authentication
- Profile retrieval and update, with auto-fill from parsed resume data
- PDF resume upload with PyPDF2 text extraction and Groq-based structured extraction
- Semantic internship matching using Sentence-Transformers embeddings and FAISS, with Groq-generated match explanations
- Multi-turn AI mock interview agent with per-answer scoring and feedback
- PDF document upload, chunked indexing, and user-isolated document-grounded Q&A
- A React frontend covering Login, Register, Dashboard, Profile, Resume, Matching, Interview, Documents, and Settings pages

12.2 In Progress

- Automated test coverage — only authentication is currently covered by pytest tests; resume, matching, interview, and document Q&A endpoints have no automated tests in the provided repository.

12.3 Pending

- DOCX resume text extraction (the upload endpoint accepts .docx files, but text extraction is implemented for PDF only)
- DOCX or other document formats for the Document Q&A feature (PDF only is currently accepted)
- Database migration history via Alembic (configured, but no migration version files are present in the provided repository)

12.4 Final Outcome

Career Match AI was successfully built as a working, end-to-end application covering resume parsing, semantic internship matching, an AI interview agent, and document-grounded Q&A, backed by a PostgreSQL (production) + FAISS data architecture and Groq-based AI services (the original development build used MySQL, ChromaDB and Gemini). Based on the reviewed source code, the application runs as an integrated system: a user can register, log in, upload a PDF resume and have it parsed, view semantically matched internships from the seeded demo dataset, practice a scored mock interview, and upload a PDF to ask grounded questions about it. The production deployment (Vercel frontend, Render backend, Cloud PostgreSQL, UptimeRobot monitoring via /health) is documented as the intended configuration and its verification is pending. What remains incomplete is broader automated test coverage beyond authentication, DOCX-format resume/document parsing, and an applied Alembic migration history.

Limitations

- Resume text extraction currently supports PDF files only; .docx resumes are accepted by the upload endpoint but are not parsed for content.
- Document Q&A currently accepts PDF files only.
- The FAISS vector indexes are local to the backend service rather than a hosted/managed vector database; index persistence on the production host (Render) should be verified.
- The Sentence-Transformers embedding model ("all-MiniLM-L6-v2") is downloaded on first run, which can add to first-time startup time.
- The internship dataset used for matching is a small, seeded demo dataset (5 postings), not a live or large-scale internship catalogue.
- CORS is currently configured to allow all origins, which the source code itself flags as a development-only setting.
- No Alembic migration version files are present in the provided repository, even though Alembic is configured; schema creation currently relies on SQLAlchemy's `create_all`.

Future Scope

- Add DOCX resume parsing support
- Expand the internship dataset beyond the current demo set
- Add automated tests for resume parsing, matching, the interview agent, and document Q&A
- Add measured evaluation of matching relevance and retrieval quality
- Support additional document formats for Document Q&A
- Introduce an applied Alembic migration history for schema changes
- Restrict CORS to specific origins and harden configuration for a production deployment
- Complete and verify the production deployment (Vercel, Render, Cloud PostgreSQL, UptimeRobot monitoring via /health) and explore further scaling (e.g., containerization, a managed vector store)

All items above are proposed future improvements and are not part of the current final implementation.

Conclusion

Career Match AI demonstrates an end-to-end application of AI techniques to a practical career-readiness problem, combining structured data extraction, retrieval-augmented generation, and conversational AI evaluation within a single system. The project integrates a React/TypeScript frontend with a FastAPI backend, and pairs PostgreSQL as the production relational source of truth with FAISS as a dedicated semantic retrieval layer, avoiding the need to force one component to serve both structured storage and vector search. The Groq API is used across the system for structured resume extraction, match explanation, interview question generation and scoring, and grounded document question answering, while Sentence-Transformers provides the underlying embeddings used for all semantic search. Together, these components allow a candidate to move from uploading a resume, to discovering relevant internships, to practicing interviews, to asking grounded questions of their own documents, within one authenticated application.